NASA Lunabotics Rover
Systems Integration & Teleoperation Guide

Prepared By

Matthew Garcia

Robotics & Embedded Systems Engineer (Systems Lead)

NASA Lunabotics Competition

Capstone Project

Spring 2026

University of Houston–Clear Lake

College of Science & Engineering

What is NASA Lunabotics?

The NASA Lunabotics Challenge is a university-level robotics competition focused on the design, integration, and operation of robotic lunar excavation systems under realistic space mission constraints.

The competition simulates engineering challenges associated with future lunar and planetary surface missions, particularly in:

- autonomous robotic systems
- remote operations
- harsh-terrain mobility
- excavation and material handling
- embedded systems reliability
- systems engineering integration

Student teams develop robotic rovers capable of operating in simulated lunar environments while applying engineering methodologies inspired by real aerospace and planetary robotics programs.

The challenge emphasizes multidisciplinary collaboration across:

- robotics
- embedded systems
- electrical engineering
- software engineering
- mechanical design
- controls engineering
- systems integration

NASA Lunabotics provides students with practical experience in:

- robotic mobility
- embedded firmware
- ROS 2 robotics middleware
- autonomous systems
- sensor integration
- communication architectures
- electromechanical validation

The competition also prepares students for careers in:

- aerospace engineering
- robotics
- embedded systems
- autonomous systems engineering.

Who We Are

UHCL Lunar Hawks

The UHCL Lunar Hawks are the official NASA Lunabotics robotics team representing the University of Houston–Clear Lake in the NASA Lunabotics competition.

The team was formed to design, integrate, and operate a robotic lunar excavation rover platform capable of functioning under realistic robotics and aerospace system constraints.

The project combines multiple engineering disciplines including:

- robotics
- embedded systems
- electrical engineering
- software engineering
- mechanical integration
- controls engineering
- systems engineering

For the UHCL Lunar Hawks rover platform, the project served as the foundation for developing a distributed robotics architecture combining:

- ROS 2 Humble
- micro-ROS
- ESP32 embedded controllers
- NVIDIA Jetson Orin NX computing
- Xbox controller teleoperation
- differential-drive mobility systems

The rover was designed using systems engineering methodologies similar to those used in modern robotic and aerospace development environments, emphasizing:

- modularity
- scalability
- subsystem integration
- real-time operational reliability

The rover architecture separates:

- high-level robotic computation
- middleware communication
- low-level embedded motor control

allowing the system to support:

- real-time teleoperation
- embedded actuator control
- future autonomy expansion
- sensor integration
- computer vision systems
- navigation stack development

The project also served as a hands-on engineering platform for developing experience in:

- distributed robotics systems
- ROS 2 middleware
- embedded firmware development
- real-time motor control
- robotic systems integration
- electromechanical validation
- field robotics operations

The UHCL Lunar Hawks rover platform was developed not only as a competition system, but also as a real-world engineering training platform designed to simulate the challenges associated with modern robotic and aerospace system development.

Table of Contents

1. Introduction

This document describes the complete software bring-up procedure used to operate the UHCL Lunar Hawks NASA Lunabotics rover using an Xbox hand controller.

The rover utilizes:

- ROS 2 Humble
- micro-ROS
- ESP32 embedded controllers
- NVIDIA Jetson Orin NX
- Xbox controller teleoperation

The software stack combines:

- high-level ROS 2 robotic communication
- embedded real-time motor control
- distributed robotics middleware
- differential-drive rover motion control

The objective of this guide is to provide a repeatable operational workflow for:

- configuring the ESP32 firmware
- building and flashing embedded firmware
- launching ROS 2 communication
- enabling Xbox controller teleoperation
- verifying rover motion through `/cmd_vel`

2. System Overview

The rover architecture consists of:

- Jetson Orin NX
  - Runs ROS 2 Humble
  - Executes teleoperation nodes
  - Hosts the micro-ROS agent
- ESP32 WROOM
  - Runs FreeRTOS + micro-ROS firmware
  - Handles PWM motor control
  - Controls rover drivetrain
- Xbox Controller
  - Publishes joystick inputs
  - Controls rover movement

3. Required Hardware

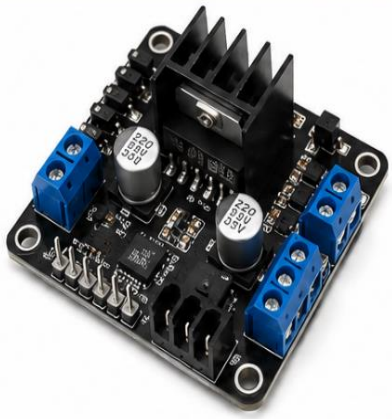
3.1 Embedded Hardware

- ESP32 WROOM
- Motor drivers
- Linear actuators
- Rover drivetrain

ESP32 WROOM

- Dual-core 32-bit Xtensa LX6 processor
- Wi-Fi 802.11 b/g/n
- Bluetooth 4.2
- 30 GPIO pins
- 3.3V operating voltage
- Ideal for micro-ROS embedded control

MOTOR DRIVERS



- Dual H-Bridge Motor Driver
- Supports DC motors up to high current
- PWM speed control
- Direction control
- Over-current and thermal protection
- Compatible with ESP32 and ROS 2 control

LINEAR ACTUATORS

- High force linear motion
- 12V / 24V options
- Extend and retract control
- Used for excavation arm, lift, and mechanisms
- Controlled via motor drivers or relays

ROVER DRIVETRAIN

- 4WD or 6WD configurations
- High torque DC gear motors
- Rugged off-road traction
- Differential drive control
- Built for rough terrain and lunar simulations

3.2 Compute Hardware

- NVIDIA Jetson Orin NX
- Ubuntu 22.04 laptop or Jetson

3.3 Teleoperation Hardware

- Xbox controller
- USB serial cable

XBOX CONTROLLER

- Wireless or USB connection
- Dual analog sticks and triggers
- Precise control for rover movement
- Plug-and-play with ROS 2
- Used for teleoperation and manual control

USB SERIAL CABLE

- USB-C to USB-A cable
- Used for serial communication
- Connects ESP32 to Jetson or laptop
- Supports data transfer and device communication
- Short length for clean setup

4. Development Environment

Operating System

`Ubuntu 22.04`

ROS Distribution

`ROS 2 Humble`

Embedded Middleware

`micro-ROS + FreeRTOS`

5. ESP32 Firmware Setup

5.1 Create Workspace

```
mkdir -p ~/uros_ws/src
cd ~/uros_ws/src
```

5.2 Clone micro_ros_setup

```
git clone -b humble https://github.com/micro-ROS/micro_ros_setup.git
```

5.3 Install Dependencies

```
cd ~/uros_ws

rosdep update
rosdep install --from-paths src --ignore-src -y
```

5.4 Build Workspace

```
colcon build
```

5.5 Source Workspace

```
source install/local_setup.bash
```

5.6 Create Firmware Workspace

```
ros2 run micro_ros_setup create_firmware_ws.sh freertos esp32
```

This generates:

```
~/uros_ws/firmware
```

---

6. Configuring micro-ROS Firmware

The rover firmware application:

```
twist_subscriber
```

subscribes to:

```
/cmd_vel
```

using:

```
geometry_msgs/msg/Twist
```

Configure the firmware for serial transport:

```
ros2 run micro_ros_setup configure_firmware.sh twist_subscriber --transport
serial --dev /dev/ttyUSB0
```

Example serial devices:

```
/dev/ttyUSB0
/dev/ttyUSB1
/dev/ttyACM0
```

---

7. Building ESP32 Firmware

Build the firmware:

```
ros2 run micro_ros_setup build_firmware.sh
```

This compiles:

- FreeRTOS
- ESP-IDF
- micro-ROS libraries
- rover motor control firmware

---

8. Flashing the ESP32

Flash the firmware onto the ESP32:

```
ros2 run micro_ros_setup flash_firmware.sh
```

After flashing:

- disconnect and reconnect ESP32 if necessary
- verify serial device appears

Check serial devices:

```
ls /dev/ttyUSB*
```

---

9. Starting ROS 2 Communication (Jetson Orin NX)

Terminal 1 — Start micro-ROS Agent

```
source /opt/ros/humble/setup.bash
source ~/uros_ws/install/local_setup.bash

ros2 run micro_ros_agent micro_ros_agent serial --dev /dev/ttyUSB0 -b 115200
-v6
```

The micro-ROS agent bridges:

- ROS 2 DDS communication
- serial UART transport
- ESP32 embedded communication

---

10. Xbox Controller Teleoperation

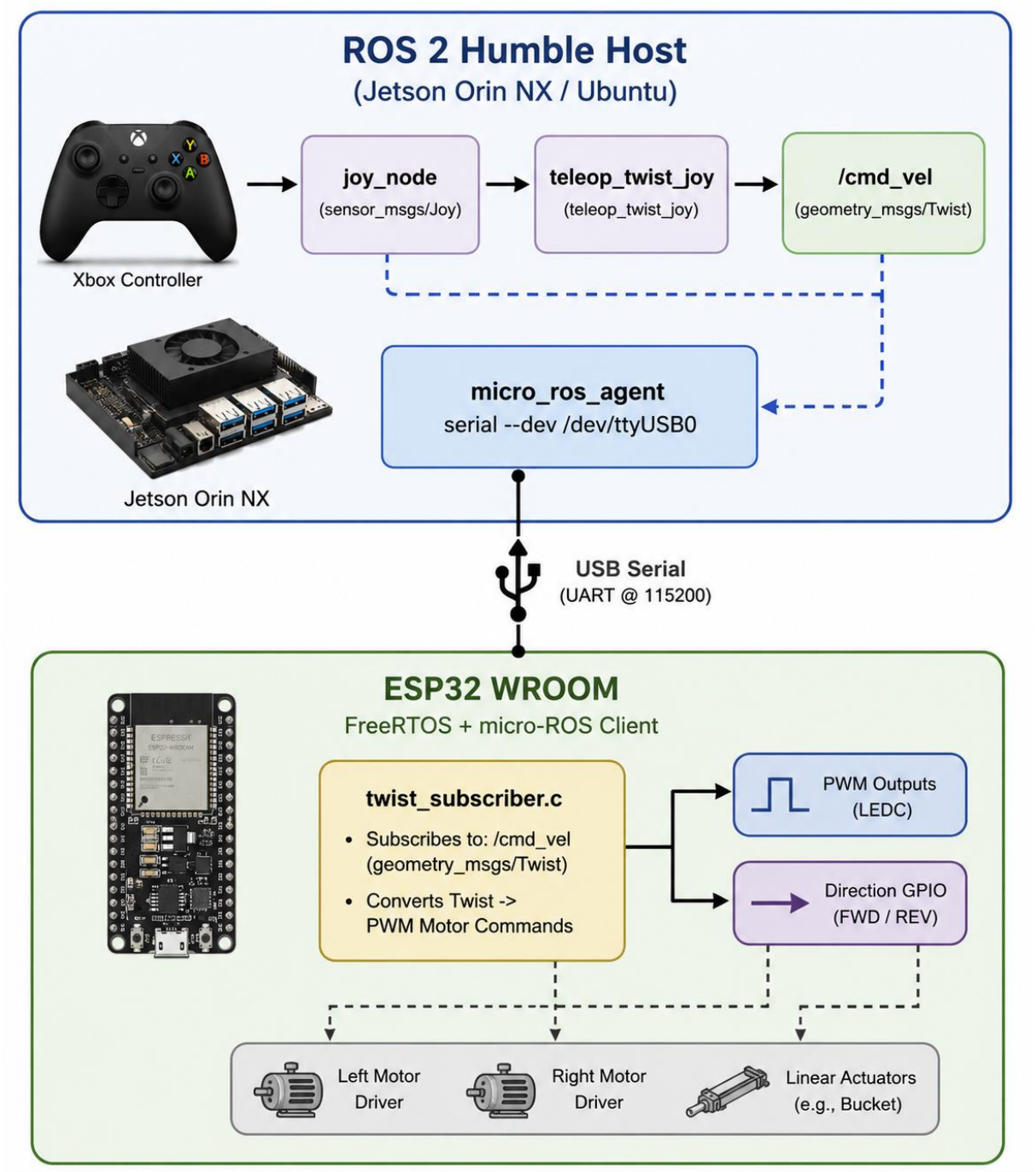
Terminal 2 — Start Xbox Controller Node

```
source /opt/ros/humble/setup.bash

ros2 run joy joy_node
```

Terminal 3 — Launch Rover Teleoperation

```
source /opt/ros/humble/setup.bash
source ~/lunabotics_ws/install/setup.bash

ros2 launch teleop launch_rover.py
```

The teleoperation pipeline is:



11. Rover Bring-Up Procedure

Follow these Steps

12. Verifying Rover Operation

Check active ROS 2 topics:

```
ros2 topic list
```

Verify velocity commands:

```
ros2 topic echo /cmd_vel
```

Check ROS 2 nodes:

```
ros2 node list
```

13. Troubleshooting

Serial Port Busy

Close:

```
idf.py monitor
```

before starting:

```
micro_ros_agent
```

ESP32 Not Detected

```
ls /dev/ttyUSB*
```

Missing /cmd_vel Messages

```
ros2 topic echo /cmd_vel
```

14. Conclusion

The NASA Lunabotics rover software stack successfully integrates:

- ROS 2 distributed robotics communication
- ESP32 embedded control
- FreeRTOS real-time firmware
- Xbox controller teleoperation
- differential-drive motion control

The architecture provides a scalable robotics platform for:

- future autonomy
- LiDAR integration
- computer vision
- navigation systems
- AI-assisted rover operation.

15. References

ROS 2 Documentation

- ROS 2 Humble Documentation
- ROS 2 Tutorials
- geometry_msgs/Twist Message Definition

micro-ROS Documentation

- micro-ROS Official Documentation
- micro_ros_setup Repository
- micro-ROS Tutorials

ESP32 & Embedded Systems Documentation

- ESP-IDF Programming Guide
- ESP32 Technical Reference Manual
- FreeRTOS Documentation

NVIDIA Jetson Documentation

- NVIDIA Jetson Orin NX Developer Kit Documentation
- JetPack SDK Documentation

Ubuntu & Linux Resources

- Ubuntu 22.04 LTS Documentation
- Ubuntu ROS Installation Guide

Teleoperation & Controller Integration

- joy ROS Package Documentation
- teleop_twist_joy Documentation

Robotics & Differential Drive Resources

- ROS Navigation Concepts
- Differential Drive Kinematics Overview

NASA Lunabotics Resources

- NASA Lunabotics Challenge
- NASA Artemis Program Overview